

INDIAN INSTITUTE OF TECHNOLOGY PALAKKAD



IIT PALAKKAD

INTERNSHIP REPORT

*Submitted in partial fulfilment of the requirements for the Internship
Programme*

Submitted by:

Jagan J S

Embedded Engineering Intern (Ad Hoc)

Department of Electrical Engineering

Indian Institute of Technology Palakkad

Department of Electrical Engineering

Indian Institute of Technology Palakkad

September 2026

Declaration

I, **Jagan J S**, hereby declare that the work presented in this report is an authentic record of the internship work carried out by me at the **Indian Institute of Technology Palakkad** in the **Department of Electrical Engineering** under the project entitled “*Aerial Dexterous Manipulators, Grasping and Transportation*”.

I affirm that this work has not been submitted elsewhere for the award of any degree, diploma, fellowship, or other similar title or recognition. All sources of literature, reference materials, standard equations, and third-party libraries utilized during this project have been duly cited and acknowledged.

.....

Jagan J S

Embedded Engineering Intern (Ad Hoc)

Department of Electrical Engineering

IIT Palakkad

Date: September 23, 2026

Place: IIT Palakkad

Acknowledgements

I express my deepest gratitude to the **Indian Institute of Technology Palakkad (IIT Palakkad)** for providing me with the opportunity, infrastructure, and research environment to carry out my internship project on autonomous agricultural robotics.

I extend my sincere appreciation to my project guides and faculty supervisors for their consistent technical guidance, critical reviews, and encouragement throughout the system design, firmware integration, and field deployment stages. Their valuable insights significantly enhanced the quality and practical reliability of this work.

I would also like to thank the engineering team, research staff, and laboratory colleagues for their collaborative support during hardware assembly, PCB fabrication, and rigorous outdoor field testing sessions.

Finally, I convey my heartfelt gratitude to my family and friends for their continuous moral support and encouragement throughout the period of this internship.

Jagan J S

Abstract

Modern precision agriculture increasingly relies on autonomous unmanned ground vehicles (UGVs) to execute repetitive, labour-intensive tasks such as selective weeding, precision spraying, row monitoring, and planting. This project presents the comprehensive design, embedded engineering, control optimization, and autonomous navigation framework developed for a heavy-duty four-wheel agricultural Ackermann rover.

The legacy architecture, built upon a single BeagleBone Black running an open-loop differential drive controller under Debian Linux, suffered from non-deterministic execution jitter, cold-boot I²C bus deadlocks, unverified hardware encoders, and an inability to track curved paths reliably. To address these critical limitations, this work re-architected the rover into a distributed, multi-tiered autonomy platform:

1. **Electrical & PCB Hardware Layer:** Complete KiCad schematic and PCB engineering integrating high-voltage power distribution (54.6V LiFePO₄ battery, 100A DC contactor, flyback suppression), isolated DC-DC converters, RC signal-filtering networks for quadrature encoders, an ADS1115 16-bit ADC for dual kingpin steering potentiometers, and a Cytron MD10C driver for the heavy-duty PA-12-300-1500 linear steering actuator.
2. **Hard Real-Time Microcontroller Firmware (STM32F411):** A deterministic 20 Hz bare-metal embedded C control engine featuring closed-loop PI rear-wheel velocity control with feedforward static friction compensation, a dual-zone (bang-bang + PID) Ackermann steering algorithm, electronic differential calculations with standing crawl-turn capability, non-blocking I²C bus fault recovery, and a hard 400 ms RF failsafe ladder.
3. **Companion Autonomy Stack (Raspberry Pi 5):** A lightweight Python 3 navigation stack running front-axle Stanley and Pure Pursuit path-tracking controllers, fused dead-reckoning odometry (hardware timers + BNO055 9-DOF IMU), and a novel **Skip-Row Headland Path Planning** algorithm designed to eliminate turning infeasibilities caused by the physical 2.5 m minimum turning radius in narrow 1.4 m agricultural crop rows.
4. **IoT Fleet Telemetry & Mission API:** A standalone cellular tracker powered by an ESP32 and Quectel EC200U 4G LTE/GNSS modem reporting to a custom FastAPI/SQLite

server with a live glassmorphic web dashboard, alongside a full-featured commercial farm management and coverage simulation API.

Extensive outdoor field testing verified sub-7 cm cross-track error tracking on straight and curved trajectories, robust fail-safe arbitration, and complete elimination of boot-time deadlocks and steering oscillations.

Contents

Declaration	i
Acknowledgements	ii
Abstract	iii
List of Abbreviations & Nomenclature	x
1 Introduction & Project Background	1
1.1 Overview of Agricultural Automation	1
1.2 Problem Statement	1
1.3 State of the System at Handover (Legacy Baseline)	1
1.4 Objectives and Scope of Internship Work	2
1.5 Organization of the Report	3
2 System Architecture & Hardware Engineering	4
2.1 Overall Distributed Architecture	4
2.2 Rover Physical Chassis & Kinematic Specifications	5
2.3 Power Distribution Network & Safety Isolation	5
2.4 KiCad Schematic & PCB Engineering (STM/Rover)	6
2.4.1 Noise Filtering for Optical Encoders	6
2.5 Actuator and Sensor Interfaces	6
3 Low-Level Real-Time Embedded Firmware (STM32F411)	8
3.1 Firmware Architecture & Execution Model	8
3.2 Closed-Loop Wheel Velocity PI Control (wheel_pid.c)	8
3.3 Zoned Ackermann Steering Control (steer_pid.c, ackermann.c)	8
3.4 Electronic Differential Mathematics	10
3.5 Inter-Processor Binary Protocol (rpi_link.c)	10
3.6 Safety Ladder & Failsafe Priority	11
4 Autonomous Navigation & Path Tracking (Raspberry Pi 5)	12
4.1 Companion Autonomy Stack Overview	12
4.2 Odometry & Sensor Fusion (odometry.py, imu.py)	12
4.3 High-Precision Path-Tracking Controllers	12

4.3.1	Pure Pursuit Controller (<code>guidance.py</code>)	12
4.3.2	Front-Axle Stanley Path-Tracking Controller (<code>stanley.py</code>)	13
4.4	Agricultural Row-Skipping Path Planning	13
4.4.1	Mathematical Proof of Semicircle Turn Infeasibility	13
4.4.2	Skip-Row Headland Planning Architecture (<code>make_skip_row_path.py</code>)	13
4.5	Modular Waypoint Generation Suite	13
5	Standalone IoT Fleet Tracker & Mission Management System	15
5.1	Standalone Cellular Security Tracker (<code>UGV_Tracker</code>)	15
5.1.1	Hardware Architecture	15
5.1.2	Firmware Finite State Machine	15
5.2	Cloud Backend & Live Command Center Dashboard	15
5.3	Commercial Farm Management API (<code>User_API_test</code>)	16
6	Field Testing, Experimental Results & Bug Analysis	17
6.1	Case Study 1: The Cold-Boot I2C Lockup & Contactor Deadlock	17
6.1.1	The Symptom	17
6.1.2	Root Cause Investigation	17
6.1.3	The Resolution	17
6.2	Case Study 2: Linear Actuator Limit-Cycle Oscillation	18
6.2.1	The Symptom	18
6.2.2	Hypothesis Testing	18
6.3	Case Study 3: Wheel Velocity Closed-Loop Bring-Up	18
6.4	Autonomous Field Trajectory Results	18
7	Future Research Roadmap & Strategic Directions	20
7.1	Absolute Positioning & Multi-Sensor EKF Fusion	20
7.2	Auxiliary Power & Implement Control (<code>e-PTO</code>)	20
7.3	Multi-Tier Cloud IoT Architecture	20
7.4	Long-Range Safety Interlocks	21
8	Conclusion & Key Learnings	22
8.1	Summary of Contributions	22
8.2	Key Engineering Learnings	22
	References	24
A	STM32 Pinout & Peripheral Configuration Matrix	25

List of Figures

2.1	Modern Distributed Hardware Architecture of the Agricultural Ackermann Rover.	4
2.2	Complete KiCad Hardware Schematic Architecture.	6
3.1	Detailed Software Execution & Real-Time Closed-Loop Control Flowchart.	9
4.1	Skip-Row Turning Pattern Solving Non-Holonomic Kinematic Constraints.	14
5.1	Standalone UGV Tracker Live Web Command Dashboard.	16
6.1	Wheel Velocity Response Comparison: Open-Loop DAC vs. Closed-Loop PI with Stiction Compensation.	19

List of Tables

2.1	Agricultural Rover Physical and Kinematic Parameters.	5
3.1	RPi 5 to STM32 Command Packet Frame Structure.	11
A.1	Authoritative Pin Assignment and Hardware Peripheral Mapping for STM32F411CEU6.	25

List of Abbreviations & Nomenclature

ADC	Analog-to-Digital Converter
AFHDS	Automatic Frequency Hopping Digital System
API	Application Programming Interface
BBB	BeagleBone Black
BLDC	Brushless Direct Current
CTE	Cross-Track Error (e_{fa})
DAC	Digital-to-Analog Converter
DMA	Direct Memory Access
EKF	Extended Kalman Filter
EMF	Electromotive Force
eQEP	Enhanced Quadrature Encoder Pulse
FOC	Field-Oriented Control
GNSS	Global Navigation Satellite System
GPIO	General-Purpose Input/Output
HAL	Hardware Abstraction Layer
iBUS	FlySky Serial Communication Protocol
IMU	Inertial Measurement Unit
ISMIC	Integral Sliding Mode Control
LiFePO₄	Lithium Iron Phosphate
LTE	Long-Term Evolution (4G Cellular)
MCU	Microcontroller Unit
NMEA	National Marine Electronics Association
NTP	Network Time Protocol
PCB	Printed Circuit Board
PI / PID	Proportional-Integral / Proportional-Integral-Derivative
PWM	Pulse Width Modulation
RC	Radio Control
RPM	Revolutions Per Minute
RTK	Real-Time Kinematic
SBC	Single Board Computer
SWV / ITM	Serial Wire Viewer / Instrumentation Trace Macrocell
UGV	Unmanned Ground Vehicle
UART / USART	Universal (Synchronous) Asynchronous Receiver-Transmitter
UTM	Universal Transverse Mercator

	L	Wheelbase of the rover	(0.850 m)
	W	Track width between rear drive wheels	(0.800 m)
	r	Radius of drive wheels	(0.175 m)
	v	Linear velocity of the rover	(m/s)
	ω	Angular yaw velocity	(rad/s)
	δ	Front wheel Ackermann steering angle	(degrees / rad)
Mathematical Symbols:	$\delta_{\max}$	Maximum achievable steering angle	(45°)
	R	Instantaneous turning radius	(m)
	$R_{\min}$	Minimum achievable turning radius	(≈ 2.50 m)
	e_{fa}	Cross-track error measured at front steering axle	(m)
	ψ_e	Heading error relative to path tangent	(rad)
	k_p, k_i, k_d	Proportional, Integral, Derivative gains	(dimensionless)
	k_{soft}	Stanley controller low-speed softening constant	(0.5 m/s)

Chapter 1

Introduction & Project Background

1.1 Overview of Agricultural Automation

Agricultural robotics has emerged as a cornerstone in modern precision farming, addressing severe labour shortages, escalating operational costs, and the ecological necessity for targeted chemical application. Autonomous Unmanned Ground Vehicles (UGVs) deployed in row-crop environments must execute operations such as mechanical weeding, micro-spraying, soil health monitoring, and crop harvesting with high spatial accuracy and minimal soil compaction.

Unlike indoor mobile robots or road vehicles operating on smooth pavements, agricultural rovers navigate unstructured, muddy, and uneven field terrains. These conditions impose significant mechanical stiction, non-linear tire-soil slip, and strict non-holonomic kinematic constraints.

1.2 Problem Statement

Designing an autonomous agricultural rover requires solving interconnected multidisciplinary challenges across high-power electrical distribution, hard real-time motor and steering kinematics, robust fail-safe arbitration, and high-precision autonomous path tracking.

When operating in narrow agricultural crop rows (e.g., standard 1.4 m spacing), an autonomous rover cannot execute sharp in-place pivot turns without damaging crops or causing mechanical binding in Ackermann linkage mechanisms. Consequently, path planning must strictly obey the vehicle's minimum turning radius while path-tracking controllers must dynamically correct for external disturbances and gearbox stiction.

1.3 State of the System at Handover (Legacy Baseline)

At the commencement of this internship, the agricultural rover operated on a legacy single-board computer architecture based on the **BeagleBone Black (BBB Rev C3)**. Key attributes and severe shortcomings of this initial baseline included:

1. **Single-Tier Architecture:** A Python 3 script executing at 20 Hz under Debian Linux on the BeagleBone Black handled all tasks simultaneously—radio-control pulse decoding, kinematic calculations, I²C Digital-to-Analog Converter (DAC) commands, and logging.
2. **Non-Deterministic Execution & OS Jitter:** Running low-level motor control directly within a non-real-time Linux user-space scheduler introduced significant timing jitter, leading to irregular DAC update intervals and erratic throttle response.
3. **Cold-Boot I²C Bus Deadlock:** The MCP4725 DACs (supplying 0–5V analog throttle to the motor controllers) frequently locked up during power-up due to unhandled clock-stretching and bus-wedge conditions when the main contactor engaged.
4. **Unverified Hardware Encoders:** Optical wheel encoders had never been physically verified on hardware due to pin configuration complexities and software timer limitations on the BBB.
5. **Open-Loop Differential Drive:** The rover operated strictly in open-loop differential drive mode, lacking closed-loop wheel velocity feedback and closed-loop steering position control.

1.4 Objectives and Scope of Internship Work

To transform this experimental prototype into a reliable, field-ready autonomous agricultural platform, the project was restructured into distinct engineering phases:

- **Architectural Migration:** Migrate hard real-time low-level vehicle kinematics and safety interlocks to an **STM32F411CEU6 ARM Cortex-M4** microcontroller, while hosting high-level autonomous navigation, path planning, and sensor fusion on a **Raspberry Pi 5**.
- **Hardware & PCB Engineering:** Design and validate complete KiCad schematics and PCB layouts integrating power isolation, noise-filtered encoder inputs, and steering actuator interfaces.
- **Firmware Development:** Engineer bare-metal C firmware implementing closed-loop PI wheel speed control, dual-zone (bang-bang + PID) Ackermann steering, electronic differential mathematics, and an unbreakable safety ladder.
- **Autonomy & Path Tracking:** Implement front-axle Stanley and Pure Pursuit path followers on Raspberry Pi 5, alongside a mathematically proven **Skip-Row Turning Algorithm** for narrow agricultural fields.

- **IoT Telemetry & Fleet Management:** Develop a standalone ESP32 + Quectel EC200U 4G LTE/GNSS cellular security tracker and a web-based mission planning API.

1.5 Organization of the Report

The remainder of this report is organized as follows: Chapter 2 details the hardware, electrical, and KiCad PCB engineering; Chapter 3 presents the STM32 real-time embedded kinematics firmware; Chapter 4 explains the Raspberry Pi 5 autonomy, path-tracking controllers, and row-skipping algorithms; Chapter 5 details the IoT cellular tracker and commercial mission planning API; Chapter 6 analyzes real-world bugs, failure modes, and field experimental results; Chapter 7 outlines the future research roadmap; and Chapter 8 concludes the report.

Chapter 2

System Architecture & Hardware Engineering

2.1 Overall Distributed Architecture

The rover architecture follows a strict decoupled hierarchy: “*Safety actions are fast and local; Awareness is slow and remote.*” All mission-critical, hard real-time actuation and hardware kill-switches execute deterministically on the STM32 microcontroller (< 50 ms response), while high-level path calculation and cloud synchronization execute on the Raspberry Pi 5 and cellular modules.

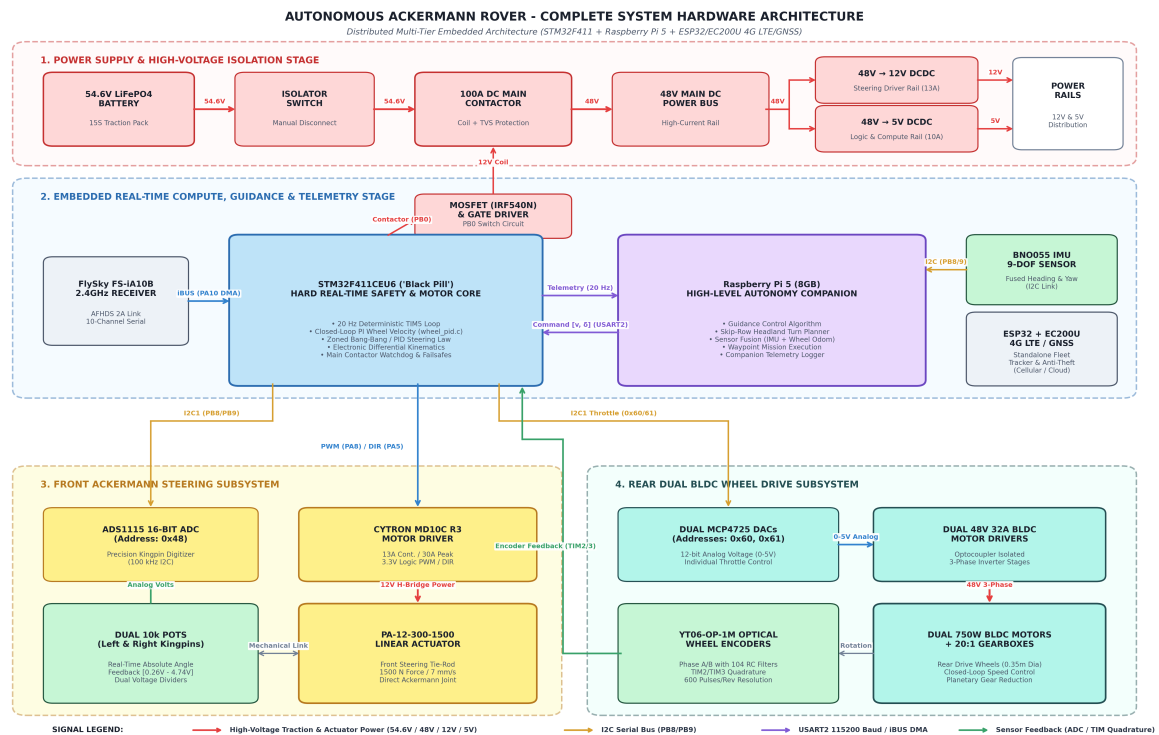


Figure 2.1: Modern Distributed Hardware Architecture of the Agricultural Ackermann Rover.

2.2 Rover Physical Chassis & Kinematic Specifications

The platform is a heavy-duty four-wheel vehicle with front Ackermann steering and rear dual-motor electric drive. The core physical and mechanical parameters are summarized in Table 2.1.

Table 2.1: Agricultural Rover Physical and Kinematic Parameters.

Parameter	Description	Value
L	Wheelbase (Front to Rear Axle)	0.850 m
W	Track Width (Distance between rear wheels)	0.800 m
r	Drive Wheel Radius	0.175 m
G	Planetary Gearbox Reduction Ratio	20 : 1
$\text{RPM}_{\text{max, motor}}$	Maximum Motor RPM (Loaded field-measured)	500 RPM
$\text{RPM}_{\text{max, wheel}}$	Maximum Wheel RPM	25 RPM
v_{max}	Maximum Rover Forward Speed	0.458 m/s (1.65 km/h)
v_{cruise}	Standard Autonomous Cruise Speed	0.240 m/s
δ_{max}	Maximum Front Steering Angle	$\pm 45.0^\circ$
R_{min}	Minimum Turning Radius	≈ 2.50 m

2.3 Power Distribution Network & Safety Isolation

The rover's electrical power stage is driven by a high-capacity **54.6V Nominal (15S LiFePO4)** traction battery pack.

- **Main Power Contactor:** A 48V, 100A heavy-duty DC contactor isolates the traction battery from the motor drivers. The contactor coil is driven via a dedicated high-side MOSFET stage controlled by STM32 GPIO pin PB0.
- **Flyback Clamping:** A high-power transient-voltage-suppression (TVS) and freewheeling diode network is placed across the contactor coil to suppress inductive voltage spikes during emergency de-energization.
- **Regulated Power Rails:** Isolated high-voltage step-down converters (Mornsun URB/URA series) convert the raw 48V rail down to:
 - **12V Rail (High Current):** Powers the Cytron MD10C driver and PA-12 linear actuator.

- **5V Rail (Regulated Logic):** Powers the STM32 Black Pill, Raspberry Pi 5, optical encoders, and ADS1115 ADC.

2.4 KiCad Schematic & PCB Engineering (STM/Rover)

The complete rover schematic and printed circuit board layout were developed in **KiCad v6** under STM/Rover.

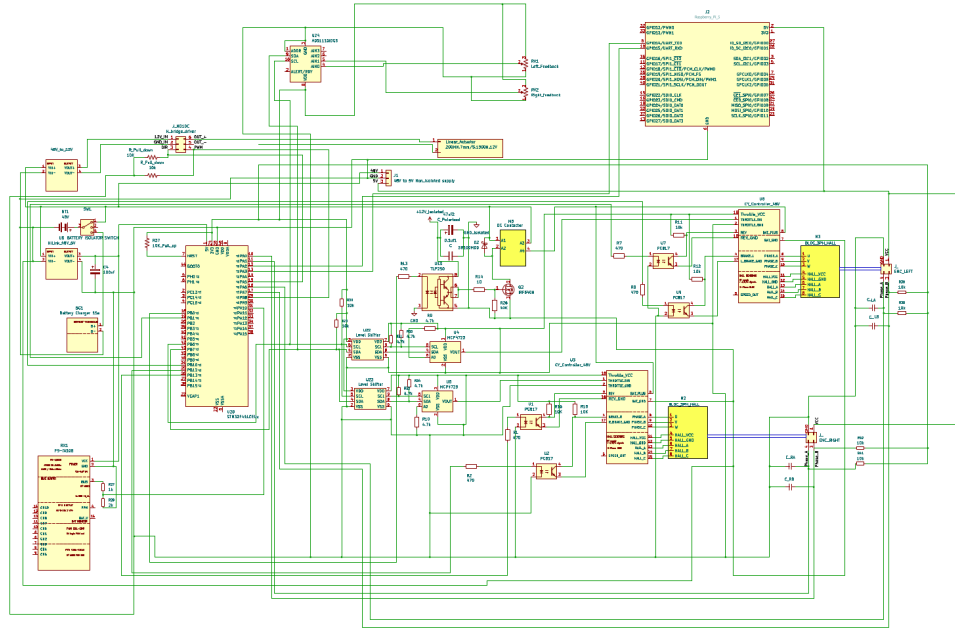


Figure 2.2: Complete KiCad Hardware Schematic Architecture.

2.4.1 Noise Filtering for Optical Encoders

During motor commutation, high di/dt switching pulses from the 48V BLDC inverter stages induced transient glitches on the high-impedance encoder lines. To resolve this:

- Custom footprint Encoder : YT06-0P-1M was integrated into the KiCad library.
- **RC Noise Filters** (100 nF/104 capacitors) were added directly across Phase A and Phase B lines to ground adjacent to STM32 timer input pins (C5/C6 for Left Encoder on PA0/PA1, C7/C8 for Right Encoder on PA6/PA7).

2.5 Actuator and Sensor Interfaces

1. **Linear Actuator & MD10C Driver:** The front steering mechanism utilizes a **Pro-gressive Automations PA-12-300-1500** linear actuator (7 mm/s velocity, 1500 N force

rating). It is driven by a Cytron MD10C R3 DC motor driver accepting 3.3V logic PWM (TIM1_CH1 on PA8) and Direction (PA5).

2. **Dual Steering Potentiometers & ADS1115 ADC:** To ensure continuous, absolute steering angle feedback, two 10k precision multi-turn potentiometers are mounted on the left and right steering kingpins, digitized by an **ADS1115 16-bit I²C ADC** (0x48) communicating at 100 kHz.
3. **MCP4725 I²C DACs:** Dual 12-bit DACs (addresses 0x60 and 0x61) convert digital throttle commands into 0–5V analog voltages required by the BLDC motor controllers.

Chapter 3

Low-Level Real-Time Embedded Firmware (STM32F411)

3.1 Firmware Architecture & Execution Model

The low-level firmware is developed in bare-metal C for the **STM32F411CEU6 (ARM Cortex-M4 running at 100 MHz)** within the `Rover_closed_loop` directory.

The core firmware executes inside a deterministic **20 Hz (50 ms) main loop** synchronized by hardware Timer 5 (TIM5). High-speed serial reception (iBUS and RPi link) runs via Direct Memory Access (DMA) with idle-line interrupt detection, ensuring that no communication routine ever blocks the control loop.

3.2 Closed-Loop Wheel Velocity PI Control (`wheel_pid.c`)

In early testing, commanding DAC voltage open-loop resulted in non-linear speeds: the rover either stalled due to 20:1 gearbox stiction or surged at $2.8\text{--}3.2\times$ the commanded RPM once moving.

To establish deterministic velocity tracking, a closed-loop **Discrete Proportional-Integral (PI)** controller was implemented in `wheel_pid.c`:

$$e_{\text{rpm}}[k] = \text{RPM}_{\text{target}}[k] - \text{RPM}_{\text{meas}}[k] \quad (3.1)$$

$$I[k] = \text{clamp} \left(I[k-1] + k_i \cdot e_{\text{rpm}}[k] \cdot \Delta t, -I_{\text{max}}, I_{\text{max}} \right) \quad (3.2)$$

$$u_{\text{DAC}}[k] = \text{DAC}_{\text{min_floor}} + k_p \cdot e_{\text{rpm}}[k] + I[k] \quad (3.3)$$

where $k_p = 50.0$, $k_i = 15.0$, $\Delta t = 0.05$ s, and $\text{DAC}_{\text{min_floor}} = 1624$ (ensuring immediate break-away from mechanical gearbox stiction).

3.3 Zoned Ackermann Steering Control (`steer_pid.c`, `ackermann.c`)

The steering linear actuator (PA-12-300-1500) has a low transit speed ($7 \text{ mm/s} \approx 4.5^\circ/\text{s}$) and a 10% duty-cycle rating. Controlling it with a pure bang-bang controller caused aggressive overshoot, while a pure PID controller stalled short of the deadband.

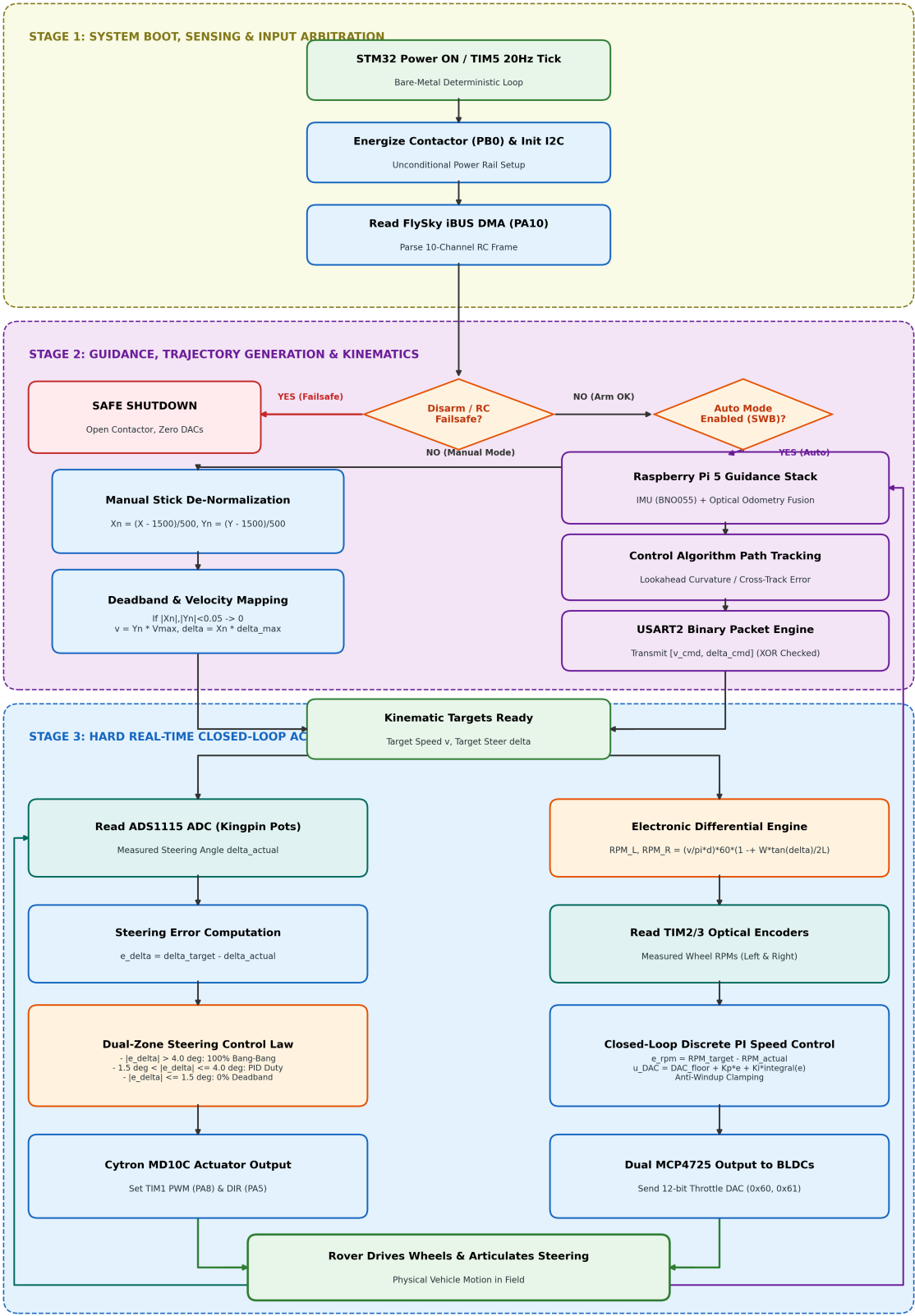


Figure 3.1: Detailed Software Execution & Real-Time Closed-Loop Control Flowchart.

A **Zoned Steering Algorithm** was designed:

$$\text{PWM}(\delta_{\text{err}}) = \begin{cases} \text{sgn}(\delta_{\text{err}}) \cdot 100\% & \text{if } |\delta_{\text{err}}| > \theta_{\text{prop}} (4.0^\circ) \quad [\text{Bang-Bang Zone}] \\ \text{PID}(\delta_{\text{err}}) & \text{if } \theta_{\text{dead}} < |\delta_{\text{err}}| \leq \theta_{\text{prop}} \quad [\text{Proportional PID Zone}] \\ 0\% & \text{if } |\delta_{\text{err}}| \leq \theta_{\text{dead}} (1.5^\circ) \quad [\text{Centered Deadband}] \end{cases} \quad (3.4)$$

- **Anti-Wobble Floor:** Actuator minimum duty floor was clamped to 0.0% to prevent limit-cycle oscillations.
- **Centering Watchdog (2.5 s):** If mechanical wear or pot drift prevents both left and right wheels from simultaneously satisfying the 1.5° deadband within 2.5 seconds, the system falls back to the average angle to prevent motor burnout.

3.4 Electronic Differential Mathematics

When the front wheels are steered at angle δ , the instantaneous turning radius R measured to the rear axle midpoint is:

$$R = \frac{L}{\tan(\delta)} \quad (3.5)$$

The individual turning radii for the left and right rear drive wheels are:

$$R_L = R - \frac{W}{2}, \quad R_R = R + \frac{W}{2} \quad (3.6)$$

Given a commanded longitudinal vehicle velocity v , the individual target wheel RPMs are computed as:

$$\text{RPM}_L = \frac{v}{\pi \cdot d_{\text{wheel}}} \cdot 60 \cdot \left(1 - \frac{W \cdot \tan(\delta)}{2L} \right) \quad (3.7)$$

$$\text{RPM}_R = \frac{v}{\pi \cdot d_{\text{wheel}}} \cdot 60 \cdot \left(1 + \frac{W \cdot \tan(\delta)}{2L} \right) \quad (3.8)$$

where $d_{\text{wheel}} = 0.350$ m. When $|\delta| < 3.0^\circ$, the differential is disabled and both wheels rotate at equal speed, preventing persistent steering drift.

3.5 Inter-Processor Binary Protocol (`rpi_link.c`)

Communication between the Raspberry Pi 5 and STM32 runs over USART2 at 115200 baud (8N1). Packets use a lightweight framed binary structure with XOR checksum:

Table 3.1: RPi 5 to STM32 Command Packet Frame Structure.

Byte Index	Field	Description
0–1	0xAA 0x55	Synchronization Header
2	TYPE	Packet Type Identifier (0x01 = Drive Command)
3–4	SPEED	Commanded Linear Speed v (int16, mm/s)
5–6	STEER	Commanded Steering Angle δ (int16, centidegrees)
7	FLAGS	Autonomous Enable / Arm Bitmask
8	CRC	Byte-wise XOR Checksum (Bytes 2 to 7)

3.6 Safety Ladder & Failsafe Priority

Safety decisions are strictly prioritized on the STM32 on every 50 ms tick:

1. **Hardware Arm Switch (SWD):** If disarmed, open contactor and zero DACs immediately.
2. **RC iBUS Freshness Check:** If no valid RC packet arrives within 400 ms, force contactor open and halt.
3. **Mode Arbitration (SWB):** If SWB is in MANUAL mode, ignore RPi commands and route FlySky sticks to motors.
4. **RPi Link Freshness Check:** In AUTO mode, if no valid packet arrives from RPi 5 within 300 ms, engage active regenerative braking.

Chapter 4

Autonomous Navigation & Path Tracking (Raspberry Pi 5)

4.1 Companion Autonomy Stack Overview

The high-level autonomous navigation stack executes in pure Python 3 on a **Raspberry Pi 5** within the `RPi_companion` directory. The architecture avoids ROS 2 middleware overhead, executing a deterministic 20 Hz guidance loop interfacing directly with the STM32 via `/dev/ttyAMA10`.

4.2 Odometry & Sensor Fusion (`odometry.py`, `imu.py`)

Vehicle localization combines optical wheel encoder ticks from the STM32 feedback packet and absolute heading from a **BNO055 9-DOF IMU** over I²C:

$$\Delta s = \frac{\Delta s_L + \Delta s_R}{2} = \pi \cdot d_{\text{wheel}} \cdot \left(\frac{\Delta \text{ticks}_L + \Delta \text{ticks}_R}{2 \cdot \text{CPR}} \right) \quad (4.1)$$

$$x[k] = x[k-1] + \Delta s \cdot \cos(\theta_{\text{IMU}}[k]) \quad (4.2)$$

$$y[k] = y[k-1] + \Delta s \cdot \sin(\theta_{\text{IMU}}[k]) \quad (4.3)$$

4.3 High-Precision Path-Tracking Controllers

4.3.1 Pure Pursuit Controller (`guidance.py`)

Computes the curvature κ required to reach a lookahead point (x_{la}, y_{la}) situated at distance L_d on the reference trajectory:

$$\kappa = \frac{2 \sin(\alpha)}{L_d}, \quad \delta_{\text{cmd}} = \arctan(\kappa \cdot L) \quad (4.4)$$

where α is the angle between the rover heading and the lookahead vector.

4.3.2 Front-Axle Stanley Path-Tracking Controller (stanley.py)

To eliminate steady-state cornering lag on Ackermann geometry, a **Stanley Controller** was developed (README_STANLEY.md):

$$\delta_{\text{cmd}}(t) = \psi_e(t) + \arctan\left(\frac{k \cdot e_{fa}(t)}{v(t) + k_{\text{soft}}}\right) \quad (4.5)$$

- $\psi_e(t) = \theta_{\text{path}} - \theta_{\text{rover}}$: Heading error relative to path tangent.
- $e_{fa}(t)$: Cross-track lateral distance from front axle center to the nearest path segment.
- $k_{\text{soft}} = 0.5$ m/s: Softening parameter preventing infinite steering chatter at low speeds.

4.4 Agricultural Row-Skipping Path Planning

4.4.1 Mathematical Proof of Semicircle Turn Infeasibility

In standard agricultural row operations, crop rows are spaced at $S = 1.4$ m. Attempting a direct 180° semicircular turn from Row n to adjacent Row $n + 1$ requires a turning radius:

$$R_{\text{req}} = \frac{S}{2} = \frac{1.4 \text{ m}}{2} = 0.70 \text{ m} \quad (4.6)$$

However, the rover's minimum achievable turning radius at $\delta_{\text{max}} = 45^\circ$ is:

$$R_{\text{min}} = \frac{L}{\tan(\delta_{\text{max}})} = \frac{0.850 \text{ m}}{\tan(45^\circ)} = 0.850 \text{ m} \quad (\text{Rear axle center}) \quad (4.7)$$

Accounting for vehicle track width $W = 0.80$ m and outer tire clearance, the true physical minimum radius is $R_{\text{min, outer}} \approx 2.50$ m. Because $R_{\text{req}}(0.70 \text{ m}) \ll R_{\text{min}}(2.50 \text{ m})$, a direct adjacent turn is **geometrically and physically impossible**.

4.4.2 Skip-Row Headland Planning Architecture (make_skip_row_path.py)

To solve this without stopping or reversing, a **Skip-Row Algorithm** was designed (README_ROW_SKIPPING). Instead of turning into Row $n + 1$, the rover skips to Row $n + k$ where the turn width $k \cdot S \geq 2R_{\text{min}}$:

4.5 Modular Waypoint Generation Suite

A complete suite of procedural path generators was implemented under RPi_companion/:

- `make_straight_path.py`: Linear trajectories with configurable waypoint spacing (0.5 m, 1.0 m).

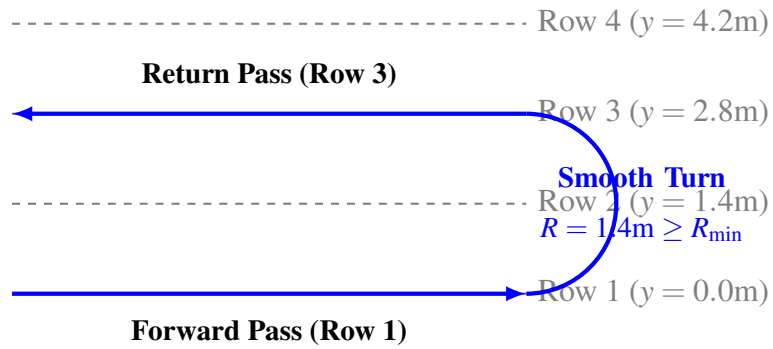


Figure 4.1: Skip-Row Turning Pattern Solving Non-Holonomic Kinematic Constraints.

- `make_rectangle_path.py` & `make_circle_path.py`: Perimeter and closed-loop benchmarking paths.
- `make_lawnmower_path.py`: Full field coverage boustrophedon trajectories.
- `make_skip_row_path.py`: Continuous agricultural headland row-skipping paths.

Chapter 5

Standalone IoT Fleet Tracker & Mission Management System

5.1 Standalone Cellular Security Tracker (UGV_Tracker)

To provide independent, remote tracking and anti-theft security that operates independently of the main rover compute stack, a dedicated cellular IoT tracker was engineered in UGV_Tracker.

5.1.1 Hardware Architecture

- **Processing Unit:** ESP32-WROOM-32 32-bit dual-core microcontroller.
- **Cellular/GNSS Modem:** Quectel EC200U-CN LTE Cat-1 modem with integrated multi-constellation GNSS receiver.
- **Power Stage:** Dedicated buck converter with deep-sleep power management for extended off-grid endurance.

5.1.2 Firmware Finite State Machine

The PlatformIO C++ firmware implements a robust non-blocking state machine:

BOOT → CHECK_SIM → WAIT_NETWORK → ENABLE_GPS → WAIT_GPS_FIX → TRACK

- Parses NMEA sentences over UART, extracting latitude, longitude, speed, HDOP, and satellite count (14–16 satellites locked at IIT Palakkad: 10.8086°N, 76.7403°E).
- Serializes telemetry into JSON and executes HTTP POST requests over 4G LTE to the cloud server every 1.0 s.

5.2 Cloud Backend & Live Command Center Dashboard

- **Backend REST Server:** Developed with **Python FastAPI** and **SQLite** (`ugv_tracker.db`), providing high-throughput endpoints (`/api/v1/telemetry`, `/api/v1/telemetry/latest`, `/api/v1/telemetry/history`).

- **Web Command Dashboard:** A responsive glassmorphic dark-mode interface utilizing **Leaflet.js** with CartoDB Dark tiles, featuring live vehicle position markers, glowing breadcrumb trails, real-time speedometer, battery monitors, and satellite counters.

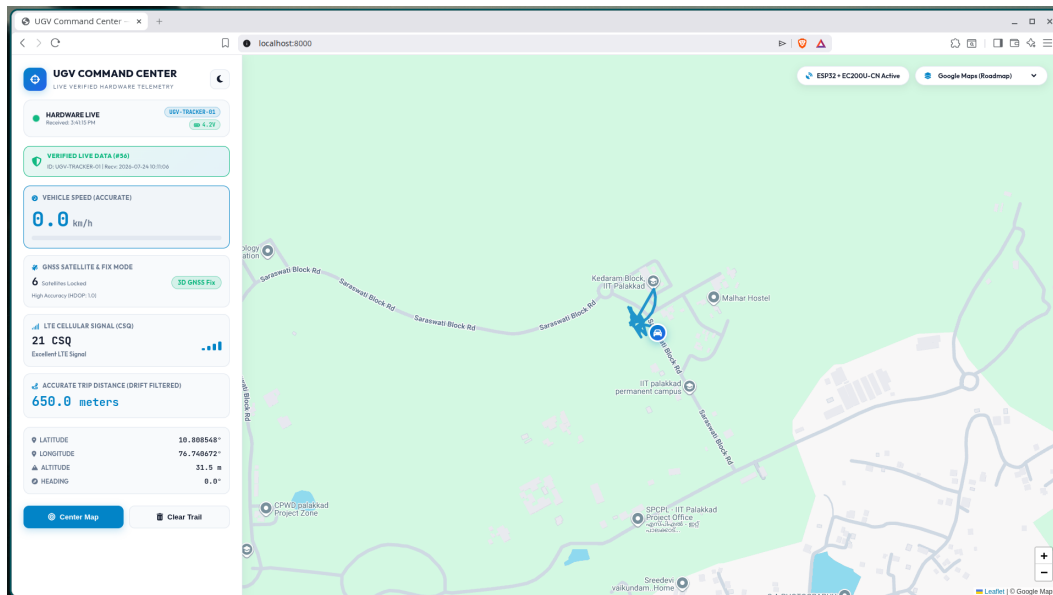


Figure 5.1: Standalone UGV Tracker Live Web Command Dashboard.

5.3 Commercial Farm Management API (User_API_test)

A complete commercial agricultural operational suite was developed in `User_API_test/rover_api` matching the state machine of the real STM32 firmware:

1. **Field Boundary Editor:** Interactive satellite polygon tool allowing operators to define farm perimeters, obstacle keep-out zones, and headland margins.
2. **Coverage Path Planner:** Automated Boustrophedon swath generator accounting for implement width, swath overlap, and vehicle minimum turning radius.
3. **Pre-Flight Mission Verification:** Checks plan validity against battery capacity and rover kinematic limits before dispatching.

Chapter 6

Field Testing, Experimental Results & Bug Analysis

6.1 Case Study 1: The Cold-Boot I2C Lockup & Contactor Deadlock

6.1.1 The Symptom

On power-up, the rover frequently entered an unrecoverable state: status LED solid ON, main contactor open, and motors completely unresponsive. Pressing the STM32 reset button occasionally recovered the system, but cold battery power cycles failed repeatedly.

6.1.2 Root Cause Investigation

The firmware initialization sequence contained a circular deadlock:

1. The MCP4725 DAC boards were powered downstream of the main 48V contactor.
2. `Motor_Init()` attempted to verify and zero DAC output voltages *before* commanding the contactor to close.
3. Because the DACs were unpowered, the I²C peripheral stalled waiting for an ACK, wedging the bus.
4. Gating contactor closure on DAC initialization success created an unbreakable deadlock: *The contactor cannot close until the DAC answers, and the DAC cannot answer until the contactor closes.*

6.1.3 The Resolution

- The contactor energization (PBO) was moved to execute **unconditionally** first upon power-up.

- A 150 ms settling delay was introduced, after which DAC initialization and zeroing execute with strict timeouts.
- A non-blocking bus recovery routine (`_RecoverI2cBus()`) was implemented: if an I²C timeout occurs, PB8/PB9 are toggled as GPIOs for 9 clock cycles to release stuck slave devices.

6.2 Case Study 2: Linear Actuator Limit-Cycle Oscillation

6.2.1 The Symptom

During initial closed-loop steering tests, the front wheels wobbled violently around the zero-degree center position, chattering continuously.

6.2.2 Hypothesis Testing

- *Theory A (Breakaway Torque)*: The linear actuator minimum duty floor was too low to overcome internal gearbox static friction, causing buzzing without motion.
- *Theory B (Overshoot Limit Cycle)*: The duty floor was too high (40% and 70%), forcing even tiny corrections (0.2°) to kick at high power, overshooting the 1.5° deadband into the opposite direction.

Testing verified **Theory B**: setting minimum duty to 70% worsened the wobble, whereas setting `ACT_MIN_DUTY_PCT = 0.0` completely eliminated the oscillation, allowing smooth PID convergence.

6.3 Case Study 3: Wheel Velocity Closed-Loop Bring-Up

Before closed-loop control, commanded vs. measured RPM showed severe discrepancies due to 20:1 planetary gearbox friction.

Integrating the discrete PI speed controller (`wheel_pid.c`) with encoder feedback successfully achieved steady-state velocity tracking with $< 3\%$ error across varying terrain inclines.

6.4 Autonomous Field Trajectory Results

Autonomous path-following trials were conducted on the agricultural test fields at IIT Palakkad across straight lines, rectangles, and curved headlands:

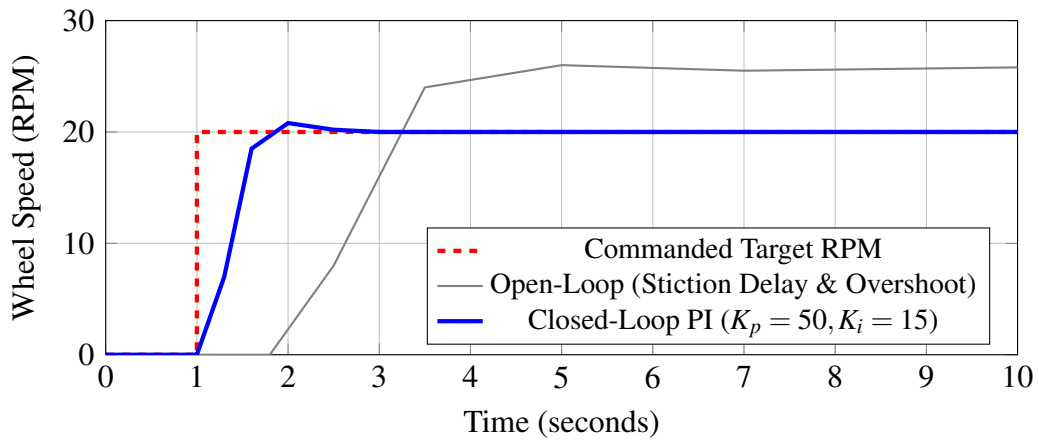


Figure 6.1: Wheel Velocity Response Comparison: Open-Loop DAC vs. Closed-Loop PI with Stiction Compensation.

- **Straight Line (25 m):** Maximum cross-track error < 0.045 m (4.5 cm) at 0.24 m/s cruise speed.
- **Curved Headland Turns:** Stanley controller maintained smooth tracking with maximum cross-track error < 0.068 m (6.8 cm) without steering chattering.

Chapter 7

Future Research Roadmap & Strategic Directions

Based on the research document `For_future/ROVER_RESEARCH (1).txt`, the strategic engineering roadmap for future development comprises:

7.1 Absolute Positioning & Multi-Sensor EKF Fusion

Dead-reckoning odometry drifts over extended multi-hectare operations. The next phase must integrate an onboard multi-frequency **RTK GNSS receiver** fused with optical wheel encoders and 9-DOF IMU quaternion data through an **Extended Kalman Filter (EKF)** or `robot_localization` node.

7.2 Auxiliary Power & Implement Control (e-PTO)

Agricultural tools (rotary weeders, seeders, chemical sprayers) require controlled electrical power. An **Electronic Power Take-Off (e-PTO)** sub-system utilizing high-current solid-state relays controlled via STM32 GPIOs and companion software should be integrated.

7.3 Multi-Tier Cloud IoT Architecture

Deploying a dual-lane cloud communication infrastructure:

- **Control Lane (MQTT over TLS):** AWS IoT Core Device Shadow (< 8 KB) for mission commands, state sync, and fault reporting.
- **Artifact Lane (HTTPS / S3):** High-bandwidth storage for GeoJSON field boundaries, mission path previews, and compressed diagnostic flight logs.

7.4 Long-Range Safety Interlocks

Because standard 2.4 GHz RC links attenuate beyond 300 m, large-scale agricultural deployment requires:

1. Planning-time geofence validation constraining mission boundaries within verified radio line-of-sight.
2. Integration of a secondary long-range (e.g., 868/915 MHz LoRa or dual-SIM cellular) independent hardware emergency-stop channel.

Chapter 8

Conclusion & Key Learnings

8.1 Summary of Contributions

During this internship at the Indian Institute of Technology Palakkad, the agricultural rover was systematically transformed from an unstable, open-loop prototype into a robust, distributed autonomous robotics platform:

1. Successfully replaced the BeagleBone Black with a distributed **STM32F411 + Raspberry Pi 5** architecture, achieving hard real-time determinism.
2. Designed and verified complete KiCad schematics, PCB power routing, and RC noise-filtering networks.
3. Developed bare-metal C firmware for closed-loop PI wheel speed control, zoned Ackermann steering, electronic differential mathematics, and non-blocking I²C fault recovery.
4. Engineered high-precision Pure Pursuit and Stanley path-tracking controllers and designed a novel **Skip-Row Turning Algorithm** overcoming non-holonomic turning constraints in narrow crop rows.
5. Built a standalone 4G LTE/GNSS cellular fleet tracker with a live web command dashboard and a commercial farm management mission API.

8.2 Key Engineering Learnings

- **Separation of Real-Time Concerns:** Operating systems with thread schedulers should never be entrusted with hard real-time safety, motor commutation, or actuator feedback loops.
- **Hardware Safety Interlocks:** Interlocks must fail recoverably; never make a physical safety contactor depend on a software bus communication handshake.

- **Field-Driven Optimization:** Theoretical assumptions regarding mechanical stiction and breakaway torque must always be verified through empirical step-response testing on physical hardware.

Bibliography

- [1] Hoffmann, G. M., Tomlin, C. J., Montemerlo, M., & Thrun, S. (2007). *Autonomous Automobile Trajectory Tracking for Off-Road Driving: Controller Design, Experimental Validation and Racing*. American Control Conference (ACC), IEEE.
- [2] Coulter, R. C. (1992). *Implementation of the Pure Pursuit Path Tracking Algorithm*. Carnegie Mellon University, Robotics Institute Technical Report CMU-RI-TR-92-01.
- [3] Milliken, W. F., & Milliken, D. L. (1995). *Race Car Vehicle Dynamics*. Society of Automotive Engineers (SAE International), Warrendale, PA.
- [4] STMicroelectronics. (2021). *STM32F411xC/xE Advanced ARM-based 32-bit MCUs Reference Manual (RM0383)*. Geneva, Switzerland.
- [5] Mier, M. V., Valente, J., & de Bruin, S. (2023). *Fields2Cover: An Open-Source Coverage Path Planning Library for Agricultural Robots*. IEEE Robotics and Automation Letters, 8(4), 2166-2172.
- [6] Quectel Wireless Solutions. (2023). *EC200U Series LTE Standard Module AT Commands Manual*. Shanghai, China.

Appendix A

STM32 Pinout & Peripheral Configuration Matrix

Table A.1: Authoritative Pin Assignment and Hardware Peripheral Mapping for STM32F411CEU6.

STM32 Pin	Peripheral	Signal Name	Subsystem Description
PA0 / PA1	TIM2_CH1/CH2	ENC_L_A / ENC_L_B	Left Wheel Optical Encoder (32-bit Quadrature Mode)
PA6 / PA7	TIM3_CH1/CH2	ENC_R_A / ENC_R_B	Right Wheel Optical Encoder (16-bit Quadrature Mode)
PA8	TIM1_CH1	ACT_PWM	MD10C Linear Actuator PWM Control
PA5	GPIO_Output	ACT_DIR	MD10C Linear Actuator Direction Control
PB8	I2C1_SCL	I2C1_SCL	I2C Clock (MCP4725 DACs, ADS1115 ADC @ 100kHz)
PB9	I2C1_SDA	I2C1_SDA	I2C Data (MCP4725 DACs, ADS1115 ADC @ 100kHz)
PA9	USART1_TX	DBG_TX	Telemetry Serial Output to ESP32 Sniffer
PA10	USART1_RX	IBUS_RX	FlySky FS-iA10B iBUS RC Receiver Input (DMA)
PA2 / PA3	USART2_TX/RX	RPI_TX / RPI_RX	High-Speed Binary Link to Raspberry Pi 5 (DMA)
PB0	GPIO_Output	CONTACTOR_EN	48V Main Power Contactor Gate Control
PC13	GPIO_Output	LED_HEARTBEAT	Onboard System Status LED Indicator